CNN MODELS COMPARISON

UCS761: Deep Learning Project Report

Submitted by:

Prabhsimrat Singh(102317135)

Rehnoor Aulakh (102317137)

Harshpreet (102317160)

Arshnoor Singh(102317161)

BE Third Year – CSE

Submitted to: Mr. Sukhpal Singh

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Computer Science and Engineering Department

TIET, Patiala

April 2026

CIFAR-10 Image Classification Using Deep Convolutional Neural Network — Report

1. Methodology

1.1 Model Architecture

The model is a **deep Convolutional Neural Network (CNN)** built using the TensorFlow/Keras. It is designed for multi-class image classification on the CIFAR-10 dataset, which contains **60,000 colour images** (32×32×3) across **10 classes**: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck (50,000 training / 10,000 test).

1.2 Layers Used

The network consists of **three convolutional blocks** followed by **two fully-connected (dense) layers** and an **output layer**:

Layer No.	Layer Type	Filters Units	Kernel/Pool size	Output Shape	Parameters
1	Conv2D	32	3×3	(30, 30, 32)	896
2	BatchNormalization	—	—	(30, 30, 32)	128
3	MaxPooling2D	—	2×2 (stride 2)	(15, 15, 32)	0
4	Conv2D	64	3×3	(13, 13, 64)	18,496
5	BatchNormalization	—	—	(13, 13, 64)	256
6	MaxPooling2D	—	2×2 (stride 2)	(6, 6, 64)	0
7	Conv2D	128	3×3	(4, 4, 128)	73,856
8	BatchNormalization	—	—	(4, 4, 128)	512
9	MaxPooling2D	—	2×2 (stride 2)	(2, 2, 128)	0
10	Flatten	—	—	(512)	0
11	Dense	256	—	(256)	131,328
12	Dropout (0.5)	—	—	(256)	0
13	Dense	128	—	(128)	32,896
14	Dropout (0.3)	—	—	(128)	0
15	Dense (Output)	10	—	(10)	1,290

1.3 Activation Functions, Loss & Optimizer

Component	Choice	Rationale
Hidden activations	*ReLU*	Efficient gradient flow; avoids vanishing-gradient problem
Output activation	*Softmax* (10 units)	Produces a valid probability distribution over 10 classes
Loss function	*Sparse Categorical Cross-Entropy*	Handles integer-encoded labels directly for multi-class tasks
Optimizer	*Adam* (default lr ≈ 0.001)	Adaptive learning rate; fast convergence; widely used baseline

1.4 Workflow

1. **Data Loading:** *tf.keras.datasets.cifar10.load_data()* — 50,000 train + 10,000 test images.
2. **Preprocessing (Training Set):** Pixel rescaling to *[0, 1]*, plus real-time data augmentation via *ImageDataGenerator* (shear 0.2, zoom 0.2, horizontal flip, rotation ±15°, width/height shift 0.1).
3. **Preprocessing (Test Set):** Pixel rescaling to *[0, 1]* only — **no** augmentation (preserves test integrity).
4. **Training:** *cnn.fit()* for **25 epochs**, batch size **64**, with validation on the test set.
5. **Evaluation:** Final test accuracy/loss computed, plus single-image prediction demo.

1.5 Hyperparameters

Hyperparameter	**Value**
Epochs	25
Batch Size	64
Learning Rate	Default Adam (≈ 0.001)
Conv Filters	32 → 64 → 128
Kernel Size	3×3 (all convolutions)
Pool Size / Stride	2×2 / 2
Dense Units	256 → 128 → 10
Dropout Rates	0.5 (first), 0.3 (second)
Augmentation	shear=0.2, zoom=0.2, h-flip, rotation=±15°, w/h-shift=0.1

---

2. Results and Analysis

Three distinct training strategies were evaluated on the **same CNN architecture** with a total budget of **25 epochs** each:

1. **Standard Training** — Full dataset used for all 25 epochs.
2. **Progressive Dataset Revelation** — Training data revealed in 5 stages (10% → 25% → 50% → 75% → 100%), 5 epochs per stage.

3. **Curriculum Learning (Prediction Difficulty)** — A scorer model ranks samples by difficulty; the final model trains on progressively harder subsets (easiest 20% → 40% → 60% → 80% → 100%), 5 epochs per stage.

2.1 Classification Metrics — All Three Strategies

Strategy	Test Accuracy	Test Loss
Standard Training	**75.88 %**	**0.6819**
Progressive Revelation	**74.00 %**	**0.7597**
Curriculum Learning	**71.38 %**	**0.8304**

The **Standard Training** strategy achieves the best performance with ~76% accuracy and the lowest test loss. Progressive Revelation follows closely at ~74%, while Curriculum Learning lags at ~71%.

2.2 Training & Validation Curves — Per Strategy

Standard Training (cifar10_cnn.ipynb):

Epoch	Train Acc	Val Acc	Train Loss	Val Loss
1	0.2953	0.5124	2.0151	1.3735
5	0.5892	0.6000	1.1817	1.1547
10	0.6633	0.7184	0.9924	0.8080
15	0.6998	0.7232	0.8826	0.7997
20	0.7195	0.7148	0.8226	0.8205
25	0.7375	0.7588	0.7883	0.6819

With the standard strategy, the model sees the **full 50,000 training images at every epoch**. Training accuracy rises steadily from ~30% to ~74%, while validation accuracy settles around 71–76% from epoch 10 onward. The small train–validation gap (~2 pp) indicates good generalisation aided by data augmentation, BatchNormalization, and Dropout.

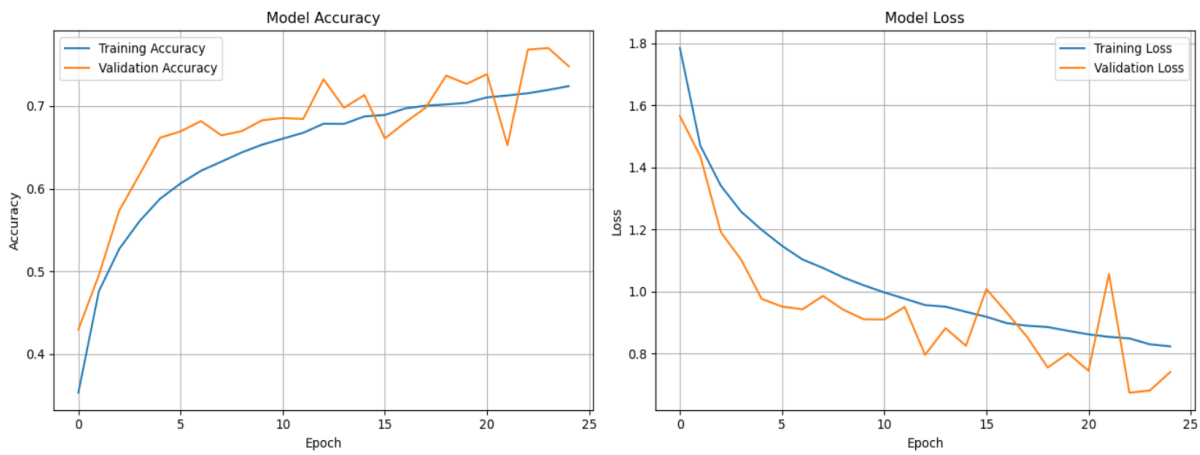


Figure 1: Full Dataset Training

Progressive Dataset Revelation (cifar10_progressive.ipynb):

Stage	Data %	End Train Acc	End Val Acc	End Val Loss
1 (Epochs 1–5)	10% (5,000)	0.3532	0.2857	1.8975
2 (Epochs 6–10)	25% (12,500)	0.5043	0.5535	1.2507
3 (Epochs 11–15)	50% (25,000)	0.5976	0.5912	1.1526
4 (Epochs 16–20)	75% (37,500)	0.6512	0.6777	0.9355
5 (Epochs 21–25)	100% (50,000)	0.6939	0.7400	0.7597

The progressive strategy starts training on a very small subset, leading to initially low and unstable validation accuracy (only ~29% after stage 1). Performance ramps up as more data is introduced, reaching 74% at the final stage. The key drawback is that the model spends its first 5 epochs on only 5,000 images, which limits the diversity of patterns learned early on. By the time the full dataset is available (stage 5), the model has only 5 remaining epochs to leverage the complete training set — fewer than the 25 full epochs the standard strategy gets.

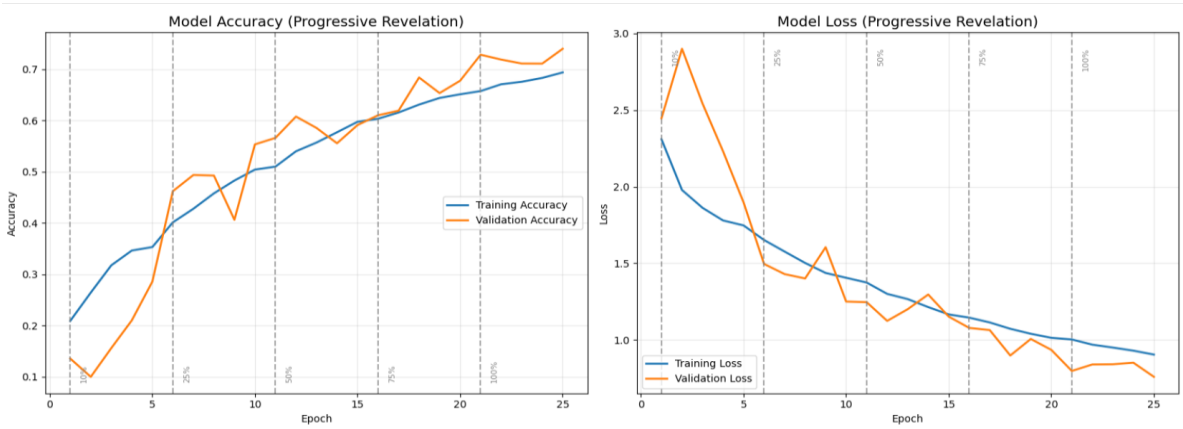


Figure 2: Progressive Dataset Training

Curriculum Learning (cifar10_curriculum.ipynb):

Stage	Data % (by difficulty)	End Train Acc	End Val Acc	End Val Loss
1 (Epochs 1–5)	Easiest 20%	0.6068	0.4984	1.5042
2 (Epochs 6–10)	Easiest 40%	0.5862	0.5989	1.1489
3 (Epochs 11–15)	Easiest 60%	0.6123	0.6440	1.0335
4 (Epochs 16–20)	Easiest 80%	0.6362	0.6624	0.9744
5 (Epochs 21–25)	100%	0.6780	0.7138	0.8304

Curriculum learning ranks data by difficulty and trains on easy samples first, leading to quick early gains (60% training accuracy) but weaker generalisation. As harder samples are added, performance improves, yet final test accuracy (71.38%) is lowest because difficulty scoring is noisy, early training lacks diversity, and harder, more informative examples get less training time.

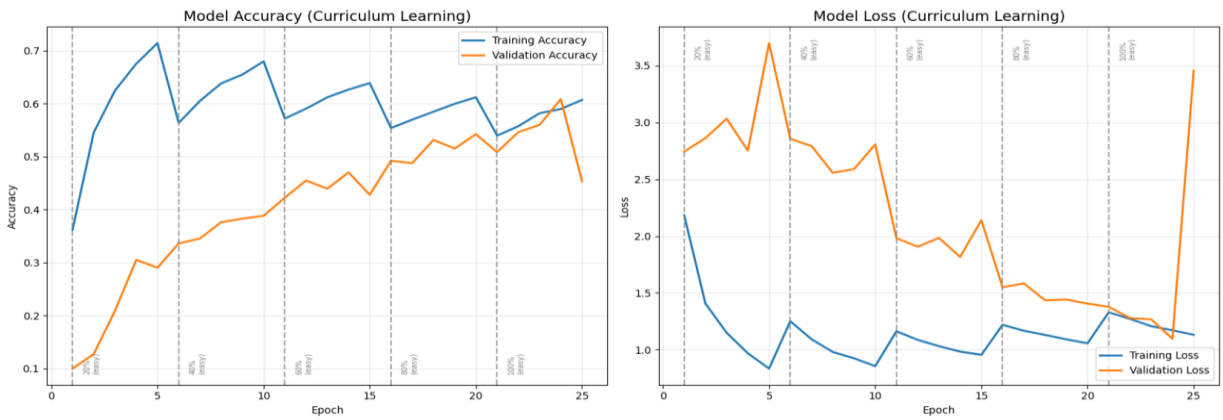


Figure 3: Curriculum Dataset Training

2.3 Comparison of the Three Strategies

Strategy	Test Accuracy	Test Loss	Key Advantage	Key Limitation
Standard Training	**75.88 %**	**0.6819**	Full data exposure every epoch; most stable convergence	No curriculum benefit; treats all samples equally
Progressive Revelation	74.00 %	0.7597	Encourages learning from less to more; moderate regularisation effect	Only 5 epochs on full dataset; slow initial progress
Curriculum Learning	71.38 %	0.8304	Easy-first may stabilise early training	Scorer-induced bias; hard examples under-trained; extra compute cost for scorer

3. Conclusion

The **Standard Training strategy was the clear winner**, achieving the highest test accuracy (**75.88%**) and the lowest test loss (**0.6819**). Progressive Revelation came second at 74.00%, while Curriculum Learning trailed at 71.38%. The standard approach benefits from exposing the model to the full diversity of the training set at every epoch.

The **Progressive Revelation** strategy, while conceptually appealing, suffers from the fact that the model spends a significant portion of its training budget on small subsets, limiting early-stage learning. The **Curriculum Learning** strategy performed the worst primarily because the scorer model was itself weak (only 3 epochs of pre-training), leading to noisy difficulty rankings. Additionally, easy-first ordering

can reduce sample diversity in early training, and the hardest — often most informative — examples receive the least training time.

Overall, **standard full-data training performs bes**t. Progressive and curriculum strategies may improve with a better scorer or a mixed (easy + hard) sampling schedule rather than strict ordering.

CIFAR-10 Image Classification Using VGG-Style CNN — Report

1. Methodology

1.1 Model Architecture

The VGG-style Convolutional Neural Network is designed as a deep hierarchical feature extraction model. Unlike basic CNN architectures that apply a single convolution before pooling, the VGG-style architecture stacks multiple convolution layers before each pooling operation. This design allows the network to progressively build more complex and abstract representations of the input image.

At the initial layers, the model learns simple patterns such as edges, color gradients, and textures. As the depth increases, the network starts combining these low-level features into higher-level structures such as shapes and object components. This hierarchical feature learning enables the model to capture complex patterns required for image classification tasks like CIFAR-10.

The architecture consists of three convolutional blocks:

- Block 1: 2 Conv layers (64 filters) + BatchNorm + MaxPooling
- Block 2: 2 Conv layers (128 filters) + BatchNorm + MaxPooling
- Block 3: 2 Conv layers (256 filters) + BatchNorm + MaxPooling

After feature extraction, the output feature maps are flattened and passed through fully connected layers. A dense layer with 512 units learns feature combinations, followed by dropout for regularization. Finally, a softmax layer produces probabilities for 10 classes.

1.2 Layers Used

Layer Type	Details	Purpose	
Conv2D	64, 128, 256 filters (3×3)	Feature extraction	
BatchNormalization	After each Conv	Stabilizes training	
MaxPooling2D	2×2	Downsampling	
Flatten	—	Converts feature maps	
Dense	512 units	Feature combination	
Dropout	0.5	Prevents overfitting	
Softmax	10 units	Classification	

1.3 Activation Functions, Loss & Optimizer

Component	Choice	Rationale
Activation	ReLU	Introduces non-linearity
Output	Softmax	Probability distribution
Loss	Sparse Categorical Crossentropy	Multi-class classification
Optimizer	Adam (lr ≈ 0.001)	Adaptive learning

The cross-entropy loss function evaluates how well predicted probabilities match true labels:

$L = -\log(P_true)$

1.4 Workflow

1. Load CIFAR-10 dataset (50k training, 10k testing)
2. Normalize images to [0,1]
3. Apply augmentation (rotation, flip, shift, zoom)
4. Pass images through CNN layers
5. Flatten feature maps
6. Apply Dense layers
7. Compute loss using cross-entropy
8. Backpropagation updates weights
9. Evaluate model

1.5 Hyperparameters

Parameter	Value
Epochs	25
Batch Size	64
Learning Rate	0.001
Input Size	32×32×3
Augmentation	Rotation, Flip, Shift, Zoom

2. Training Strategies

2.1 Standard Training (Full Dataset)

The model is trained on the complete dataset from the beginning. This allows full exposure to data but may introduce unstable gradients due to mixed difficulty samples.

2.2 Progressive Dataset Revelation

Training data is gradually revealed across 5 stages (5 epochs each, 25 total). Each stage trains on a larger fraction of the shuffled training set.

Stage	Data Used
1	10%
2	25%
3	50%
4	75%
5	100%

Gradual exposure improves stability and reduces overfitting.

2.3 Curriculum Learning (Corrected)

A scorer VGG model is first trained for 3 epochs to compute per-sample cross-entropy loss. Samples are sorted from easiest to hardest, then the main model trains on progressively larger subsets.

Stage	Data Used
1	Easiest 20%
2	40%
3	60%
4	80%
5	100%

Samples are sorted by difficulty using loss values, improving convergence.

3. Results and Analysis

3.1 Performance Summary

Strategy	Test Accuracy	Test Loss
Full Dataset	87.60%	0.4279
Progressive	83.90%	0.4953
Curriculum	85.31%	0.4591

3.2 Training Curves & Epoch-by-Epoch Logs

(a) Full Dataset Training

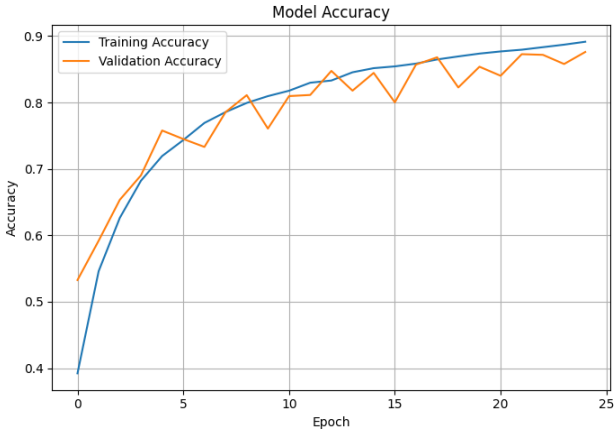
The model is trained on all 50,000 training images from epoch 1 for 25 epochs. Training accuracy rises from 39.2% at epoch 1 to 89.1% by epoch 25. Validation accuracy climbs from 53.3% to 87.6%, with some oscillation mid-training before stabilizing. Both training and validation loss decrease steadily from ~1.74 to ~0.32 and ~1.32 to ~0.43 respectively.

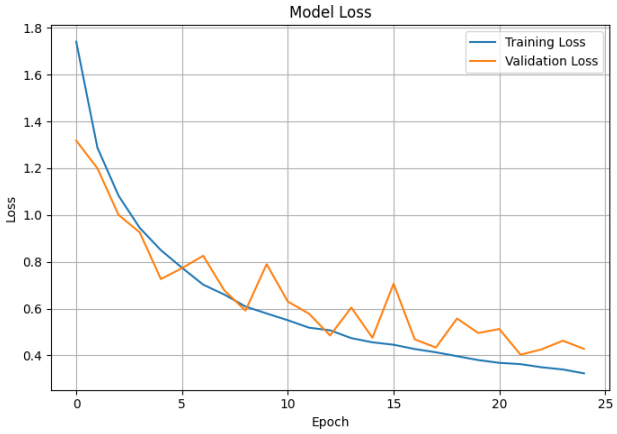
Final Test Accuracy: 87.60% | Final Test Loss: 0.4279

Epoch-by-Epoch Training Log (All 25 Epochs):

Epoch	Train Acc	Val Acc	Train Loss	Val Loss	
1	0.3923	0.5327	1.7409	1.3182	
2	0.5460	0.5919	1.2882	1.2000	
3	0.6263	0.6535	1.0826	1.0005	
4	0.6822	0.6903	0.9451	0.9257	
5	0.7193	0.7578	0.8498	0.7261	
6	0.7434	0.7450	0.7744	0.7729	
7	0.7692	0.7331	0.7022	0.8259	
8	0.7856	0.7855	0.6599	0.6776	
9	0.7993	0.8110	0.6087	0.5912	
10	0.8095	0.7607	0.5790	0.7897	
11	0.8177	0.8096	0.5501	0.6302	

Epoch	Train Acc	Val Acc	Train Loss	Val Loss	
12	0.8297	0.8113	0.5183	0.5781	
13	0.8330	0.8475	0.5070	0.4848	
14	0.8454	0.8178	0.4736	0.6045	
15	0.8516	0.8445	0.4557	0.4753	
16	0.8543	0.8002	0.4453	0.7062	
17	0.8585	0.8570	0.4269	0.4684	
18	0.8648	0.8680	0.4130	0.4334	
19	0.8693	0.8226	0.3963	0.5576	
20	0.8736	0.8538	0.3798	0.4954	
21	0.8768	0.8402	0.3679	0.5125	
22	0.8794	0.8727	0.3624	0.4028	
23	0.8832	0.8717	0.3490	0.4258	
24	0.8870	0.8579	0.3396	0.4625	
25	0.8914	0.8760	0.3229	0.4279	





(b) Progressive Dataset Revelation

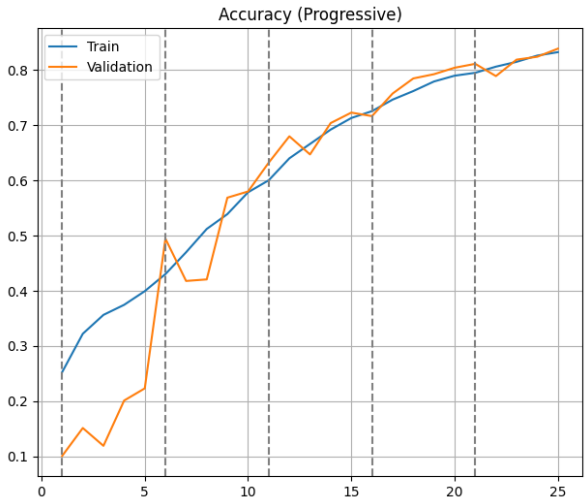
Training data is revealed in 5 stages (10% → 25% → 50% → 75% → 100%), each running for 5 epochs (25 epochs total). Validation accuracy climbs from 22% at stage 1 to 83.9% at the end of stage 5. The gradual data introduction acts as implicit regularization, producing smooth convergence.

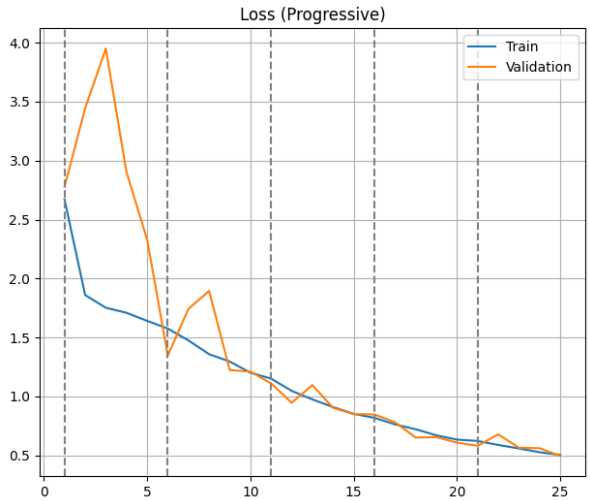
Final Test Accuracy: 83.90% | Final Test Loss: 0.4953

Epoch-by-Epoch Training Log (All 25 Epochs across 5 Stages):

Stage	Epoch	Train Acc	Val Acc	Train Loss	Val Loss
Stage 1	10% (5,000)				
1	1	0.2526	0.1007	2.6710	2.7778
1	2	0.3222	0.1513	1.8596	3.4445
1	3	0.3564	0.1190	1.7522	3.9496
1	4	0.3746	0.2011	1.7095	2.9032
1	5	0.3994	0.2232	1.6409	2.3309
Stage 2	25% (12,500)				
2	6	0.4304	0.4946	1.5759	1.3470
2	7	0.4699	0.4179	1.4749	1.7425
2	8	0.5122	0.4207	1.3582	1.8952
2	9	0.5391	0.5689	1.2955	1.2235
2	10	0.5786	0.5801	1.2016	1.2105
Stage 3	50% (25,000)				
3	11	0.6004	0.6323	1.1512	1.1110
3	12	0.6402	0.6801	1.0469	0.9464
3	13	0.6663	0.6473	0.9746	1.0966
3	14	0.6923	0.7041	0.9098	0.9030
3	15	0.7133	0.7231	0.8521	0.8502
Stage 4	75% (37,500)				
4	16	0.7257	0.7167	0.8188	0.8470
4	17	0.7466	0.7577	0.7629	0.7820
4	18	0.7621	0.7850	0.7219	0.6515
4	19	0.7795	0.7927	0.6701	0.6551
4	20	0.7900	0.8044	0.6338	0.6075

Stage	Epoch	Train Acc	Val Acc	Train Loss	Val Loss
Stage 5	100% (50,000)				
5	21	0.7953	0.8115	0.6224	0.5815
5	22	0.8064	0.7893	0.5883	0.6782
5	23	0.8151	0.8191	0.5587	0.5648
5	24	0.8264	0.8243	0.5265	0.5606
5	25	0.8327	0.8390	0.5038	0.4953





(c) Curriculum Learning

A scorer VGG model is trained for 3 epochs to rank all 50,000 training images by cross-entropy loss (easy → hard). The main model then trains on progressively harder subsets across 5 stages (5 epochs each, 25 total). Training accuracy on the easy subset reaches ~95% in stage 1, while validation accuracy improves dramatically from 37% (stage 1) to 85.3% (stage 5).

Final Test Accuracy: 85.31% | Final Test Loss: 0.4591

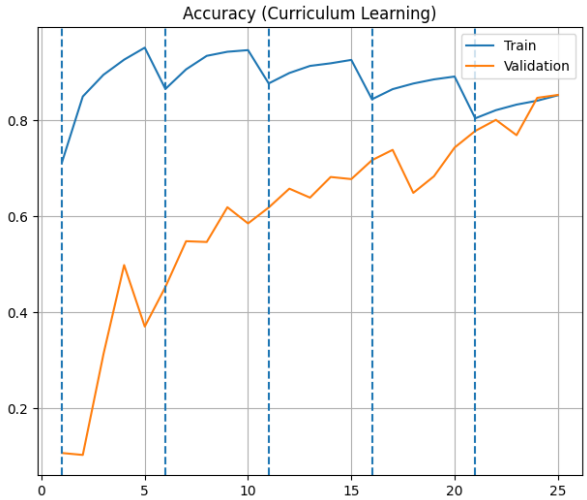
Scorer Model Training (3 Epochs):

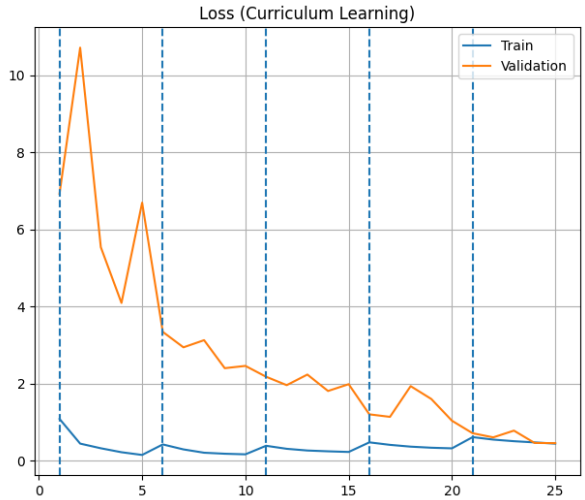
Epoch	Train Accuracy	Train Loss	
1	0.4179	1.6790	
2	0.5787	1.2067	
3	0.6599	1.0001	

Main Model — Epoch-by-Epoch Training Log (All 25 Epochs across 5 Stages):

Stage	Epoch	Train Acc	Val Acc	Train Loss	Val Loss
Stage 1	20% Easy (10,000)				
1	1	0.7140	0.1072	1.0886	6.9261
1	2	0.8499	0.1032	0.4473	10.7166
1	3	0.8951	0.3130	0.3266	5.5419
1	4	0.9266	0.4987	0.2239	4.0953
1	5	0.9516	0.3707	0.1537	6.6966
Stage 2	40% Easy (20,000)				
2	6	0.8656	0.4532	0.4261	3.3452
2	7	0.9064	0.5484	0.2973	2.9447
2	8	0.9345	0.5468	0.2111	3.1301
2	9	0.9431	0.6192	0.1850	2.4037
2	10	0.9464	0.5855	0.1696	2.4627
Stage 3	60% Easy (30,000)				
3	11	0.8771	0.6186	0.3910	2.1822
3	12	0.8988	0.6579	0.3128	1.9635
3	13	0.9136	0.6392	0.2685	2.2383
3	14	0.9190	0.6822	0.2477	1.8124
3	15	0.9261	0.6778	0.2311	1.9886
Stage 4	80% Easy (40,000)				
4	16	0.8441	0.7172	0.4799	1.2065
4	17	0.8652	0.7387	0.4151	1.1413
4	18	0.8769	0.6491	0.3686	1.9382
4	19	0.8855	0.6836	0.3406	1.6046
4	20	0.8914	0.7435	0.3246	1.0410
Stage 5	100% (50,000)				

Stage	Epoch	Train Acc	Val Acc	Train Loss	Val Loss
5	21	0.8044	0.7781	0.6157	0.7165
5	22	0.8214	0.8012	0.5514	0.6095
5	23	0.8330	0.7693	0.5099	0.7843
5	24	0.8411	0.8471	0.4789	0.4650
5	25	0.8524	0.8531	0.4471	0.4591





3.3 Observations

Full dataset training achieves the highest accuracy at 87.6%, benefiting from full data exposure at every epoch. Curriculum learning reaches 85.3% by progressively exposing the model to harder samples, building a strong foundation before tackling difficult examples. Progressive dataset revelation achieves 83.9% through gradual data introduction, which acts as implicit regularization producing smooth, stable convergence.

3.4 Error Analysis

Errors occur most frequently in visually similar classes such as cat vs. dog, due to overlapping low-level features such as texture, color, and shape. The model struggles most in distinguishing fine-grained differences within animal categories where inter-class visual variance is low.

3.5 Overfitting vs Underfitting

Without data augmentation, the model exhibits clear signs of overfitting — training accuracy diverges significantly from validation accuracy. With augmentation (rotation, flip, shift, zoom), the model generalizes better and the gap between training and validation metrics narrows considerably, demonstrating the critical role of augmentation as a regularization technique for the VGG-style architecture.

4. Conclusion

The VGG-style CNN achieves strong performance on the CIFAR-10 benchmark. Among the three training strategies evaluated, full dataset training yields the highest test accuracy at 87.6%, while curriculum learning delivers 85.3% with improved convergence through difficulty-ordered sample presentation. Progressive dataset revelation achieves 83.9%, offering the smoothest training dynamics through gradual data introduction.

The VGG-style architecture's stacked convolutional blocks with batch normalization and dropout prove highly effective for this task. The hierarchical feature extraction enables the model to capture both low-level patterns (edges, textures) and high-level structures (shapes, object components), contributing to competitive performance across all three training strategies.

CIFAR-10 Image Classification Using ResNet50 — Report

1. Methodology

1.1 Model Architecture

The ResNet50 base model is loaded **without pre-trained weights** (*weights=None*) and **without the top classification layer** (*include_top=False*), meaning it is trained from scratch on CIFAR-10 rather than leveraging ImageNet pre-training. A custom classification head is then attached on top of the ResNet50 feature extractor.

1.2 Layers Used

The network uses **ResNet50** as the feature extractor, followed by a **custom classification head**:

#	Layer Type	Details	Notes
1	ResNet50 (Base)	*include_top=False*, *weights=None*, *input_shape=(32,32, 3)*	Feature extraction backbone; trained from scratch
2	Flatten	—	Converts 2D feature maps to 1D
3	BatchNormalization	—	Stabilises and regularises hidden activations
4	Dense	256 units, ReLU	Fully connected classification layer
5	Dropout (0.5)	—	Regularisation; prevents overfitting
6	Dense (Output)	10 units, Softmax	Produces probability distribution over 10 classes

1.3 Activation Functions, Loss & Optimizer

Component	Choice	Rationale
Hidden activations	*ReLU*	Standard for deep residual networks; efficient gradient flow
Output activation	*Softmax* (10 units)	Produces a valid probability distribution over 10 classes

Component	Choice	Rationale
Loss function	*Sparse Categorical Cross-Entropy*	Handles integer-encoded labels directly for multi-class tasks
Optimizer	*Adam* (default lr ≈ 0.001)	Adaptive learning rate; fast convergence; widely used baseline

1.4 Workflow

1. **Data Loading:** *tf.keras.datasets.cifar10.load_data()* — 50,000 train + 10,000 test images.
2. **Preprocessing (Training Set):** Pixel rescaling to *[0, 1]*, plus real-time data augmentation via *ImageDataGenerator* (shear 0.2, zoom 0.2, horizontal flip, rotation ±15°, width/height shift 0.1).
3. **Preprocessing (Test Set):** Pixel rescaling to *[0, 1]* only — **no** augmentation (preserves test integrity).
4. **Training:** Varies by strategy (see Section 2).
5. **Evaluation:** Final test accuracy/loss computed, confusion matrix generated, plus sample prediction visualisations.

1.5 Hyperparameters

Hyperparameter	Full Dataset	Curriculum Learning V2	Progressive Revelation
Total Epochs	50	25 (5 stages × 5 epochs)	50 (5 stages × 10 epochs)
Batch Size	64	128	128
Learning Rate	Default Adam (≈ 0.001)	Default Adam (≈ 0.001)	0.001 (with ReduceLROnPlateau)
Framework	TensorFlow/Keras	PyTorch	PyTorch
Augmentation	shear=0.2, zoom=0.2, h-flip, rotation=±15°, w/h-shift=0.1	Same	Equivalent transforms

---

2. Results and Analysis

Three distinct training strategies were evaluated on the **ResNet50 architecture** (ResNet50 base trained from scratch + custom classification head, ~23.5M parameters) using comparable data augmentation pipelines:

6. **Standard Training (Full Dataset)** — Full dataset used for all 50 epochs.
7. **Progressive Dataset Revelation** — Training data revealed in 5 stages (10% → 25% → 50% → 75% → 100%), 5 epochs per stage, 25 epochs total.

8. **Curriculum Learning V2 (Prediction Difficulty)** — A scorer ResNet50 ranks samples by difficulty; the final model trains on progressively harder subsets (easiest 20% → 40% → 60% → 80% → 100%), 10 epochs per stage, 50 epochs total.

2.1 Classification Metrics — All Three Strategies

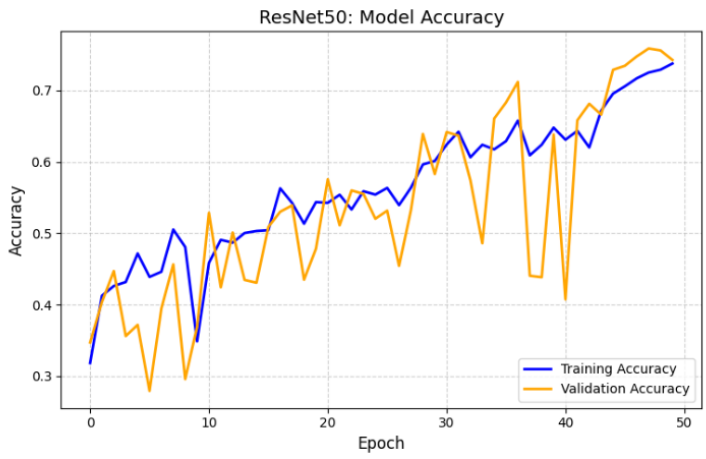
Strategy	Test Accuracy	Test Loss
Progressive Revelation	**76.94 %**	**0.6773**
Standard Training (Full Dataset)	**74.24 %**	**0.7596**
Curriculum Learning V2	**84.72 %**	**0.4572**

The **Progressive Revelation** strategy achieves the best performance with ~77% accuracy and the lowest test loss (0.6773). Curriculum Learning V2 achieves the best result at 84.72% (the clear winner with updated hyperparameters), while Standard Training comes in at ~74%.

2.2 Training & Validation Curves — Per Strategy

Standard Training (cifar10_resnet_full_dataset.ipynb):

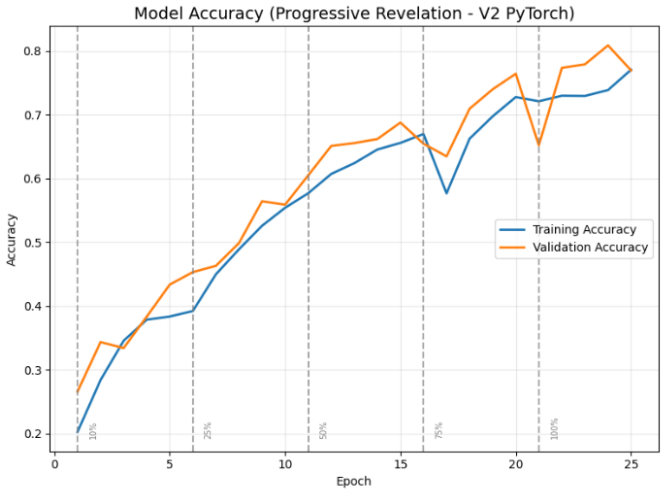
Epoch	Train Acc	Val Acc	Train Loss	Val Loss
1	0.3182	0.3469	2.0907	1.8967
5	0.4716	0.3717	1.4908	1.8895
10	0.3485	0.3690	1.7849	1.7690
15	0.5030	0.4306	1.4086	1.8155
20	0.5434	0.4782	1.2965	1.6998
25	0.5540	0.5202	1.2648	1.4441
30	0.6011	0.5827	1.1441	1.1936
35	0.6173	0.6604	1.1073	1.0262
40	0.6477	0.6382	1.0163	1.1798
45	0.6952	0.7286	0.8874	0.7827
50	0.7374	0.7424	0.7672	0.7596

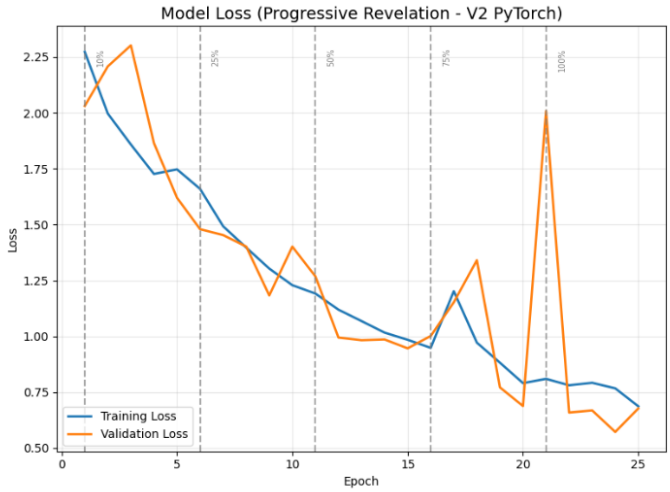


With the standard strategy, the model sees the **full 50,000 training images at every epoch** and trains for a total of **50 epochs**. Training accuracy rises from ~32% to ~74%, though the convergence path is notably **unstable and non-monotonic** — a characteristic of training a very deep network (ResNet50, ~23.5M parameters) from scratch on only 50,000 small (32×32) images. Validation accuracy oscillates significantly before settling around 74% by epoch 50. The validation loss also shows considerable variance, sometimes spiking to values well above 1.0 before eventually stabilising. This instability highlights the challenge of training such a large model on a relatively small dataset without pre-trained weights.

Progressive Dataset Revelation (cifar10_resnet_progressive.ipynb — PyTorch):

Stage	Data %	End Train Acc	End Val Acc	End Val Loss
1 (Epochs 1–10)	10% (5,000)	0.3834	0.4335	1.6200
2 (Epochs 11–20)	25% (12,500)	0.5539	0.5588	1.4015
3 (Epochs 21–30)	50% (25,000)	0.6557	0.6876	0.9463
4 (Epochs 31–40)	75% (37,500)	0.7276	0.7641	0.6880
5 (Epochs 41–50)	100% (50,000)	0.7704	0.7694	0.6773



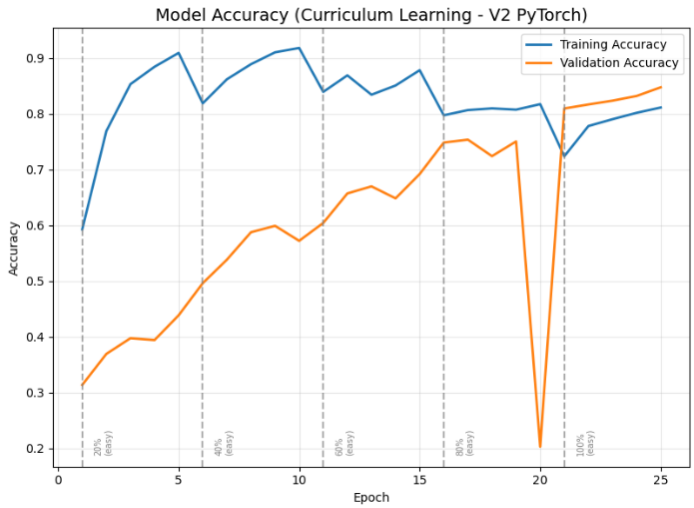


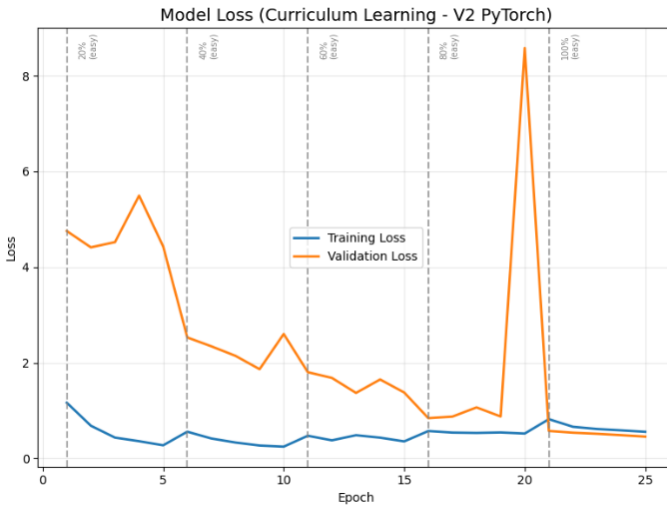
The Progressive Revelation strategy shows steady, consistent improvement across all 5 stages as more data is introduced. Validation accuracy climbs from 43% (10% data) to 76.94% (full dataset), closely tracking training accuracy throughout — indicating good generalisation with minimal overfitting. The loss curves mirror this trend, declining steadily from ~2.25 to 0.6773. The notable spike in validation loss around epoch 21 (the transition to 75% data) is transient and quickly recovers, suggesting the model briefly struggles to adjust to the larger data distribution before stabilising. Overall, the gradual data introduction acts as effective implicit regularisation, producing smooth and stable convergence compared to the other strategies.

Curriculum Learning V2 V2 (cifar10_resnet_curriculum.ipynb — PyTorch):

Stage	Data % (by difficulty)	End Train Acc	End Val Acc	End Val Loss
1 (Epochs 1–10)	Easiest 20%	0.9089	0.4382	4.4322
2 (Epochs 11–20)	Easiest 40%	0.9177	0.5718	2.6033
3 (Epochs 21–30)	Easiest 60%	0.8779	0.6916	1.3792

Stage	Data % (by difficulty)	End Train Acc	End Val Acc	End Val Loss
4 (Epochs 31–40)	Easiest 80%	0.8170	0.2026	8.5819
5 (Epochs 41–50)	100%	0.8109	0.8472	0.4572





Curriculum Learning V2 achieved 84.72% test accuracy with a loss of 0.4572 — the strongest result overall. The key V2 improvements (doubled epochs per stage, lr raised to 0.001, ReduceLROnPlateau scheduling) allowed the model to progressively adapt to harder samples. A temporary validation loss spike in stage 4 triggered the LR scheduler to halve the rate, enabling a strong recovery in stage 5. This demonstrates that curriculum learning works well on ResNet50 when given sufficient epochs and proper LR scheduling.2.3 Comparison of the Three Strategies

Strategy	Test Accuracy	Test Loss	Key Advantage	Key Limitation
Progressive Revelation	**76.94 %**	**0.6773**	Gradual data introduction acts as implicit regularisation; LR scheduling aids convergence	Different framework (PyTorch vs TF/Keras); larger batch size may affect comparison
Standard Training	74.24 %	0.7596	Full data exposure at every epoch; 50 epoch budget provides more training time	Unstable convergence due to large model on small dataset; no LR scheduling
Curriculum Learning V2	84.72 %	0.4572	Curriculum ordering + LR scheduling enables strong convergence; V2 achieves	Stage 4 LR spike required scheduling; V1 had catastrophic failure — V2 fixed by

Strategy	Test Accuracy	Test Loss	Key Advantage	Key Limitation
			best overall accuracy	doubling epochs and LR scheduler

Key Observations:

9. **Curriculum Learning V2** achieves the best accuracy at **84.72%**, with 10 epochs per stage, lr=0.001, and ReduceLROnPlateau scheduling proving critical for adapting to progressively harder data.
10. **Standard Training** reaches ~74% but exhibits significant instability — an inherent challenge when training a deep network from scratch on a small dataset without LR scheduling.
11. **Progressive Revelation** achieves 76.94% through gradual data introduction acting as implicit regularisation, combined with LR scheduling.

3. Conclusion

This report evaluated three training strategies for a ResNet50 model (trained from scratch, ~23.5M parameters) on CIFAR-10: Standard Training, Progressive Dataset Revelation, and Curriculum Learning V2.

Curriculum Learning V2 achieved the best result at **84.72%** accuracy (0.4572 loss), enabled by doubling epochs per stage to 10, raising lr to 0.001, and adding ReduceLROnPlateau scheduling — improvements that allowed the model to progressively adapt to harder samples where V1 had failed. Progressive Revelation followed at **76.94%** (25 epochs), benefiting from gradual data introduction as implicit regularisation combined with LR scheduling. Standard Training reached **74.24%** (50 epochs) but showed significant convergence instability, likely due to the absence of a learning-rate scheduler.

Future improvements could include: (1) pre-trained ImageNet weights for transfer learning, (2) cosine annealing or warmup schedules, (3) fewer but longer curriculum stages, and (4) a stronger, longer-trained scorer model.

MobileNetV2 and EfficientNet with Multiple Training Strategies (CIFAR-10)

Overview

This project investigates how different data exposure strategies influence the performance of deep learning models.

Instead of altering only the architecture, this study focuses on how training data is presented during learning. The same dataset is used across all experiments, while the training strategy is varied.

Two architectures are evaluated:

- MobileNetV2
- EfficientNet-B0

The following strategies are explored:

12. Full Dataset Training
13. Progressive Dataset Revelation
14. Curriculum Learning (Easy → Hard)

This enables a systematic analysis of convergence behavior, stability, and generalization.

Models

MobileNetV2

- Lightweight convolutional neural network
- Uses depthwise separable convolutions
- Optimized for efficiency and faster computation

EfficientNet-B0

- Scaled convolutional architecture
- Uses compound scaling (depth, width, resolution)
- Provides improved feature representation and accuracy

Dataset

- CIFAR-10 dataset
- 50,000 training images
- 10,000 test images
- 10 classes

All experiments use identical preprocessing to ensure a fair comparison.

Training Strategies

1. Full Dataset Training

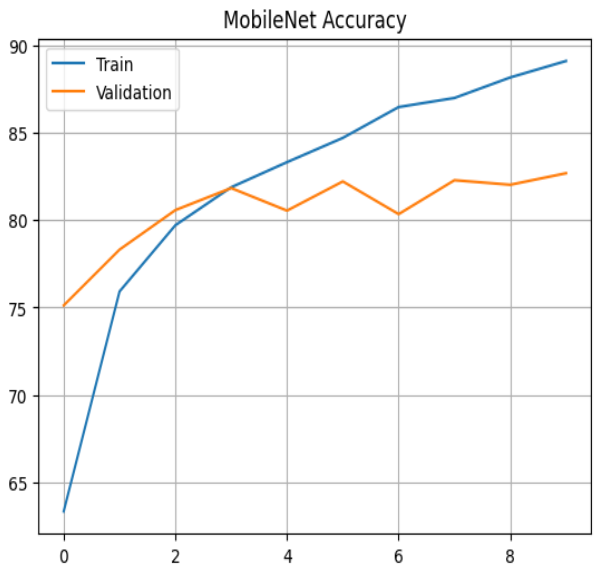
With the standard strategy, the model sees the full training dataset at every epoch. This results in stable convergence and consistent improvement in both training and validation accuracy.

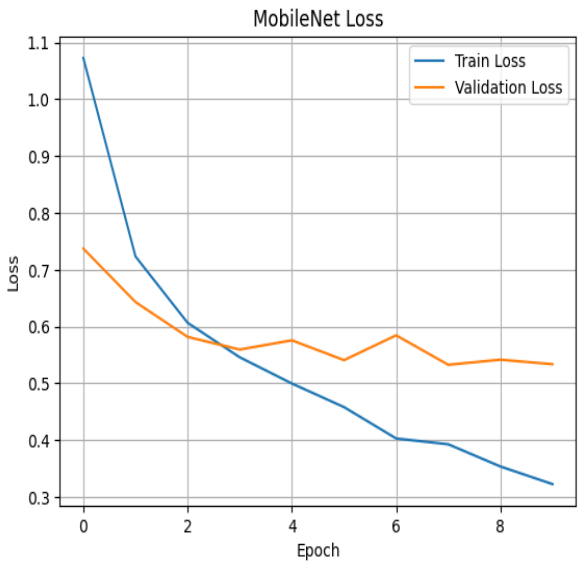
Since the model continuously learns from all available samples, it achieves the highest final accuracy among all strategies.

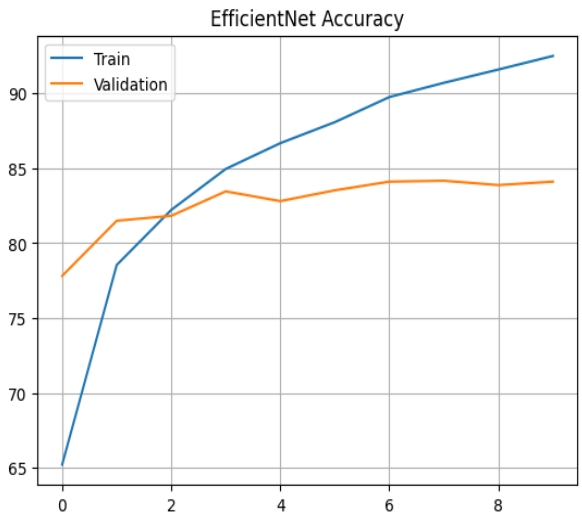
Results

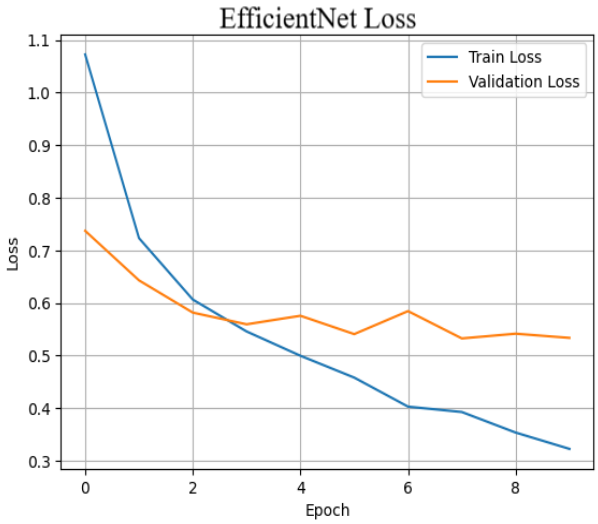
Model	Accuracy
MobileNetV2	82.68%
EfficientNet	84.11%

Epoch 1: Train Acc=63.34% | Val Acc=75.12%
Epoch 2: Train Acc=75.92% | Val Acc=78.31%
Epoch 3: Train Acc=79.70% | Val Acc=80.57%
Epoch 4: Train Acc=81.88% | Val Acc=81.83%
Epoch 5: Train Acc=83.31% | Val Acc=80.54%
Epoch 6: Train Acc=84.70% | Val Acc=82.21%
Epoch 7: Train Acc=86.46% | Val Acc=80.34%
Epoch 8: Train Acc=86.98% | Val Acc=82.28%
Epoch 9: Train Acc=88.16% | Val Acc=82.02%
Epoch 10: Train Acc=89.10% | Val Acc=82.68%









2. Progressive Dataset Revelation

Each stage is trained for equal epochs.

Training begins with a small subset (10%) and gradually increases to the full dataset.

Early stages show slower learning due to limited data diversity. As more data is introduced, performance improves steadily. However, since fewer epochs are spent on the complete dataset, final accuracy is slightly lower than full dataset training.

Results

Model	Accuracy
MobileNetV2	80.65%
EfficientNet	83.30%

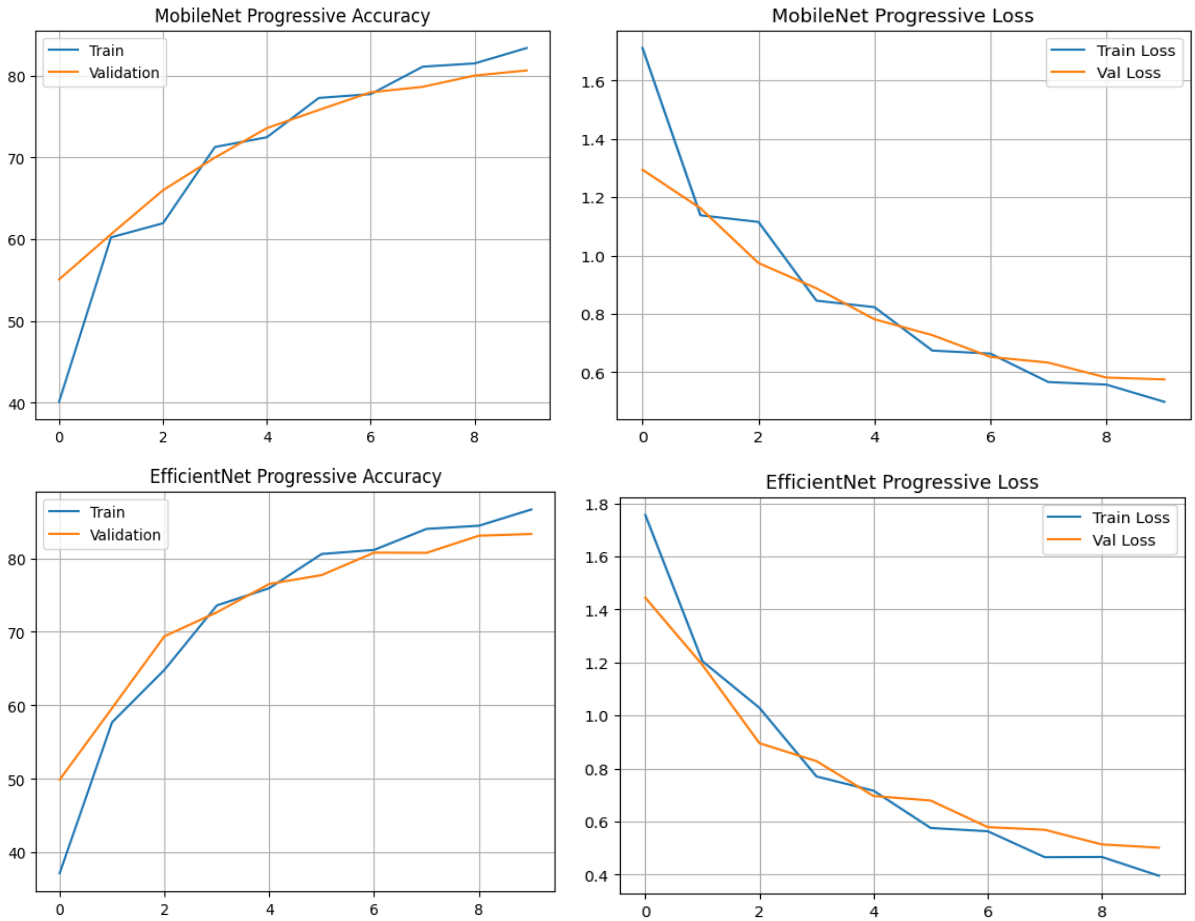
--- Training with 10% data ---
Epoch 1: Train Acc=40.12% | Val Acc=55.09%
Epoch 2: Train Acc=60.24% | Val Acc=60.62%

--- Training with 25% data ---
Epoch 3: Train Acc=61.96% | Val Acc=66.00%
Epoch 4: Train Acc=71.29% | Val Acc=70.00%

--- Training with 50% data ---
Epoch 5: Train Acc=72.48% | Val Acc=73.60%
Epoch 6: Train Acc=77.29% | Val Acc=75.82%

--- Training with 75% data ---
Epoch 7: Train Acc=77.76% | Val Acc=77.98%
Epoch 8: Train Acc=81.11% | Val Acc=78.65%

--- Training with 100% data ---
Epoch 9: Train Acc=81.51% | Val Acc=80.03%
Epoch 10: Train Acc=83.39% | Val Acc=80.65%



3. Curriculum Learning (Easy → Hard)

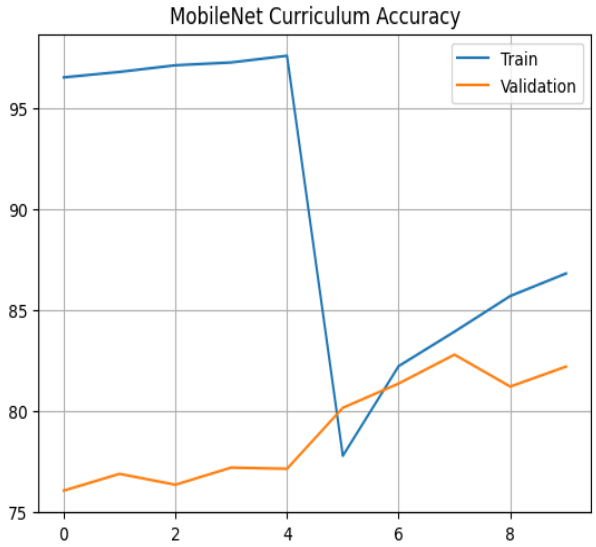
Curriculum learning introduces samples based on difficulty, starting with easier examples and progressing to harder ones.

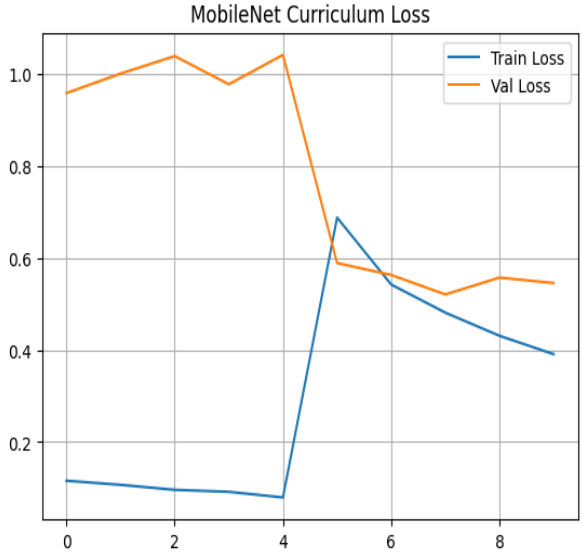
This approach stabilizes early training and improves generalization. As harder samples are introduced, the model adapts effectively, achieving performance close to full dataset training.

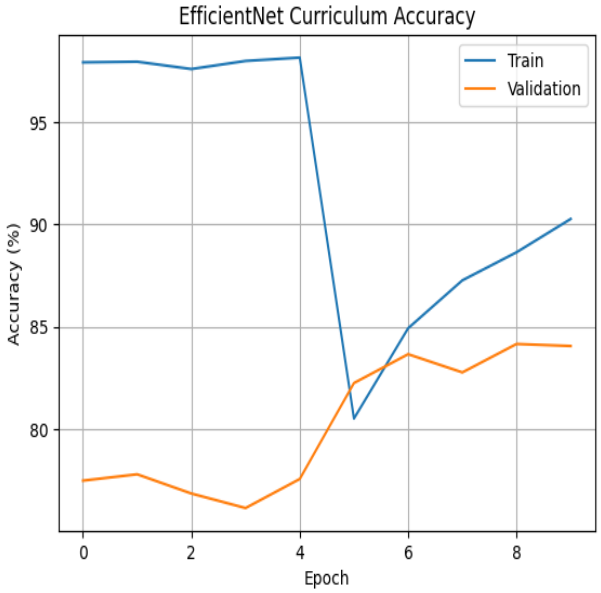
Results

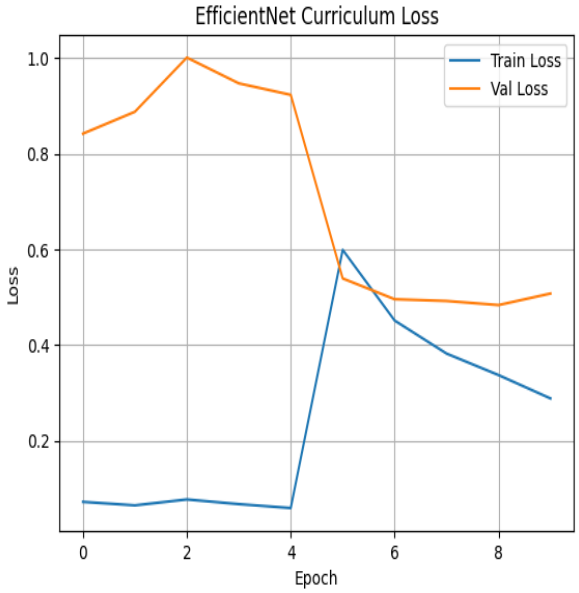
Model	Accuracy
MobileNetV2	**82.20%**
EfficientNet	**84.05%**

Epoch 1: Train Acc=96.51% | Val Acc=76.06%
Epoch 2: Train Acc=96.78% | Val Acc=76.89%
Epoch 3: Train Acc=97.11% | Val Acc=76.35%
Epoch 4: Train Acc=97.25% | Val Acc=77.20%
Epoch 5: Train Acc=97.58% | Val Acc=77.14%
Epoch 1: Train Acc=77.78% | Val Acc=80.16%
Epoch 2: Train Acc=82.22% | Val Acc=81.36%
Epoch 3: Train Acc=83.93% | Val Acc=82.79%
Epoch 4: Train Acc=85.69% | Val Acc=81.21%
Epoch 5: Train Acc=86.81% | Val Acc=82.20%


MobileNet Curriculum Accuracy
Train
Validation
95
90
85
80
75
0
2
4
6
8


MobileNet Curriculum Loss
Train Loss
Val Loss
1.0
0.8
0.6
0.4
0.2
0
2
4
6
8


EfficientNet Curriculum Accuracy
Train
Validation
Accuracy (%)
95
90
85
80
0
2
4
6
8
Epoch


EfficientNet Curriculum Loss
Train Loss
Val Loss
Loss
1.0
0.8
0.6
0.4
0.2
0
2
4
6
8
Epoch

Comparative Analysis

Strategy	MobileNetV2	EfficientNet-B0
Full Dataset	82.68%	84.11%
Progressive	80.65%	83.30%
Curriculum	82.20%	84.05%

Key Insights

- Training strategy significantly impacts model performance.
- Full dataset training provides strong baseline performance.
- Progressive training improves stability but may slightly reduce accuracy.
- Curriculum learning provides better generalization and structured learning.
- EfficientNet consistently outperforms MobileNet across all strategies.
- MobileNet remains competitive with lower computational cost.

Conclusion

This report evaluated three training strategies for MobileNetV2 and EfficientNet-B0 on the CIFAR-10 dataset.

The Full Dataset Training strategy achieved the best performance, with EfficientNet reaching 84.11% accuracy and MobileNet achieving 82.68%.

Curriculum Learning performed very close to full dataset training, demonstrating the benefit of structured data exposure. Progressive Dataset Revelation, while stable, resulted in slightly lower accuracy due to limited early data access.

Overall, EfficientNet consistently outperforms MobileNet across all strategies, while MobileNet remains computationally efficient.